Reviewer Pack — Homotopy Group Rank Inverse Proportional to Disjointness Com…

Entry c0f03be34ae3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 00:01:42 UTC

Homotopy Group Rank Inverse Proportional to Disjointness Communication Complexity

Entry ID: `c0f03be34ae3` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-11 00:01:42 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Homotopy Theory) **Field B** (complexity object): Disjointness Communication Complexity

Statement:

> For any bipartite graph G = (U ∪ V, E) with |U| = |V| = n, the rank of the first homotopy group $\pi_1(\Gamma\_G)$ of its communication matrix Γ_G is Θ(1/CC(G)), where CC(G) is the deterministic communication complexity of disjointness on G.

Rationale (proposer's reasoning):

> Homotopy invariants capture global structural constraints of communication matrices, which may correlate with inherent complexity barriers. The fundamental group's rank reflects non-trivial loops in the matrix's topology, potentially mirroring the information-theoretic cost of coordination.

Taxonomy category: `COMM_COMPLEXITY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `e42572112ed978d5`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_bipartite_graph(n):
        U = list(range(n))
        V = list(range(n, 2*n))
        E = set()
        for u in U:
            for v in V:
                if random.choice([True, False]):
                    E.add((u, v))
        return (U, V, E)

    def clique_incidence_matrix(G):
        n = len(G[0])
        M = [[0] * (2*n) for _ in range(2*n)]
        for u in G[0]:
            for v in G[1]:
                if (u, v) in G[2]:
                    M[u][v+n] = 1
                    M[v+n][u] = 1
        return M

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        rank = 0
        for j in range(n):
            i_max = -1
            for i in range(rank, m):
                if A[i][j]:
                    i_max = i
                    break
            if i_max == -1:
                continue
            A[rank], A[i_max] = A[i_max], A[rank]
```

```
            for i in range(m):
                if i != rank and A[i][j]:
                    factor = Fraction(A[i][j], A[rank][j])
                    for k in range(n):
                        A[i][k] -= factor * A[rank][k]
            rank += 1
        return rank

    def communication_complexity(G):
        n = len(G[0])
        lower_bound = math.ceil(math.log2(n))
        return lower_bound

    n = random.choice([5, 10, 15, 20, 30, 40])
    G = generate_bipartite_graph(n)
    M = clique_incidence_matrix(G)
    rank = gaussian_elimination(M)
    cc = communication_complexity(G)

    return {
        "metric_name": "homotopy_group_rank",
        "metric_value": rank,
        "instances_tested": 1,
        "conjecture_holds": rank == Fraction(1, cc),
        "counterexample": "" if rank == Fraction(1, cc) else f"n={n}, rank={rank}, CC(G)={cc}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif sum(1 for r in results if not r["conjecture_holds"]) / len(results) >= 0.2:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a1218782.py", line 86, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a1218782.py", line 68, in run_trial
    M = clique_incidence_matrix(G)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a1218782.py", line 37, in clique_incide
    M[u][v+n] = 1
    ~~~~^^^^^
IndexError: list assignment index out of range
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed with IndexError, preventing data collection. Support condition cannot be evaluated without successful trials. | next: Fix the clique_incidence_matrix implementation to handle index bounds properly

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	37340
2	preregistration	ollama_remote	qwen3:8b	0	0	26363
3	novelty	ollama_remote	qwen3:8b	0	0	20839
4	novelty	ollama_remote	qwen3:8b	0	0	14230
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11031
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10244
7	judge	ollama_remote	qwen3:8b	0	0	14902

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 134948 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/c0f03be34ae3.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c0f03be34ae3.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c0f03be34ae3.tar.gz` (if generated)